

dgaiot 技术白皮书

本体驱动 AI 编程 • DLAS 架构 • 确定性工业物联网

dgaiot 项目组

2026 年 7 月

目录

1	核心设计原则	3
1.1	本体是唯一真相源	3
1.2	生产环境运行确定性代码	3
1.3	修正本体，而非修正代码	3
2	DLAS 架构	3
2.1	五层架构	3
2.2	影子设备：物理世界 1:1 映射	4
2.3	TDengine 存储结构	4
2.4	DLAS 数据流	5
3	本体驱动 AI 编程	5
3.1	闭环流程	5
3.2	本体 → 代码示例	5
4	授权模型	6
4.1	三层 MQTT 授权	6
5	验证结果	6
5.1	全链路测试	6
6	结论	6

1 核心设计原则

1.1 本体是唯一真相源

本体（Ontology）定义了工业系统的完整语义模型——从物理设备到业务规则的映射关系。在 dgaIoT 架构中，本体包括：

- 物模型（Thing Model）：125 个 Modbus 测点定义，含地址、类型、阈值
- 产品模板（ProductTemplet）：设备类型、属性、事件、服务
- 四层拓扑（Site/Gateway/Device/Point）：物理世界到数字孪生的映射
- 通道配置（Channel）：协议适配器与 TDengine 数据库映射
- ACL 规则：设备授权（ProductSecret/DeviceSecret）与用户角色权限

1.2 生产环境运行确定性代码

开发时 AI 编程（读本体 → 写代码）。

运行时确定性执行（gen_statem · MQTT · TDengine）。

没有 AI 推理，没有概率，没有黑盒。

每一步可追溯、可回放、可审计。

1.3 修正本体，而非修正代码

本体是严肃的——企业标准、合规要求、物理规律。代码是派生的。

- 日志显示温度 82°C 未告警 → 检查 gen_statem 是否收到消息（运行时问题）
- 再检查 condition 是否正确编译（AI 生成问题）
- 最后才怀疑本体（极少）
- 本体 = 宪法，不常修。代码 = 法规，本体派生，可迭代。

2 DLAS 架构

2.1 五层架构

```

EDGE:      iotStudio (Python+Vue) — DeviceAccess(9) → StreamEngine(15)
SECURITY:  auth • role • ACL/CLP • Hooks • JWT
ACTION:    Shadow(gen_statem) • Bridge • MQTT • Rule Engine
LOGIC:     Ontology Engine • Model Registry • Reasoner • Policy
           Optimizer
DATA:      Parse/PG(23类) • TDengine(_{ChannelId}._{ProductId}) • EMQX
           (:1883)

```

2.2 影子设备：物理世界 1:1 映射

每个物理设备对应一个 Erlang OTP gen_statem 进程：

```

init() -> {ok, authenticate, Device}.
authenticate(cast, heartbeat) -> {next_state, online, Device}.
online(cast, {data, Props}, Device) ->
    evaluate(Rules, Props), %% 物模型编译的规则
    bridge -> Parse + TDengine,
    case alarm_triggered of
        true  -> {next_state, alarm, Device};
        false -> {keep_state, Device}
    end.
alarm(cast, heartbeat, Device) ->
    {next_state, online, Device}. %% 恢复

```

211 台设备的数字孪生 = 211 个 Erlang 进程，总内存消耗 < 1 MB。

2.3 TDengine 存储结构

```

-- 宏: ?Database(Name) = "_" ++ Name
-- 宏: ?Table(Name)    = "_" ++ Name

CREATE DATABASE IF NOT EXISTS _85ef6b7459 KEEP 365;
CREATE STABLE IF NOT EXISTS _85ef6b7459._2de1b3e1b8
    (ts TIMESTAMP, value FLOAT, quality INT)
    TAGS (devaddr NCHAR(50));
CREATE TABLE ... USING _2de1b3e1b8 TAGS('02110120089');

INSERT INTO _85ef6b7459.sub_2de1b3e1b8_02110120089
    VALUES (NOW, 2.35, 192);

```

2.4 DLAS 数据流

```
Modbus(:502) -> Edge Collector -> MQTT
-> EMQX -> Shadow gen_statem -> evaluate(Rules)
-> Parse(update status) + TDengine(INSERT telemetry)
```

3 本体驱动 AI 编程

3.1 闭环流程

1. 企业需求：合规、SLA、安全策略、业务规则
2. 本体建模：thing_model、ProductTemplet、Channel、ACL
3. AI 生成代码：.erl (gen_statem + rules)、.py (bridge)、SQL (schema)
4. 测试验证：单元测试 (state transitions)、集成测试 (MQTT chain)、日志验证 (TDengine INSERT)
5. 确定性生产：gen_statem evaluate、dgaiot_tdengine:INSERT、MQTT publish
6. 日志反馈：错误日志 → 修正 AI 生成逻辑；性能日志 → 优化参数

3.2 本体 → 代码示例

Listing 1: thing_model.json 片段

```
{
  "name": "油压",
  "dataForm": {"address": "40300", "protocol": "modbus",
               "originaltype": "float32_AB"},
  "identifier": "oil_pressure"
}
```

AI 自动生成:

```
%% Modbus: readHoldingRegisters(40300, 2, float32_AB)
%% MQTT: dgaiot/oil_field_01/gw_131/rtu_001/oil_pressure/data
%% TDengine: INSERT INTO _85ef6b7459._2de1b3e1b8 VALUES (NOW, v, 192)
%% gen_statem:
online(cast, {data, #{oil_pressure := V}}, Device)
  when V > 3.0 ->
```

```
{next_state, alarm, Device, [{state_timeout, 30000,
    heartbeat_missed}]]};
```

4 授权模型

4.1 三层 MQTT 授权

5 验证结果

5.1 全链路测试

6 结论

dgaiot 提出了一种**本体驱动 AI 编程**的工业物联网架构。本体是唯一真相源, AI 在开发阶段将本体编译为确定性代码, 生产环境运行的是 gen_statem + MQTT + TDengine 的确定性管道, 不包含任何 AI 推理或概率决策。DLAS 五层架构已通过 Kylin-DMZ 实测试验证。

表 1: 设备/用户/超级用户三层授权

层级	ClientID	认证方式	权限范围
设备授权	{PID}_{DevAddr}	ProductSecret/DeviceSecret	报遥测
用户授权	{Token}{Type}	Session Token + Role ACL	订阅/下发
超级用户	dgiot (127.0.0.1)	本地免检	内部通信

表 2: 端到端验证矩阵

测试项	方法	结果
MQTT pub/sub	Kylin-DMZ 实机	3 发 3 收, DLAS topic 格式
gen_statem Shadow TDengine 存储	Erlang 编译运行 源码对齐建表	init→online→alarm→online INSERT 3 行, SELECT 3 行
dlink 设备授权	ClientID={PID}_{DevAddr}	全部 OK